

HACKER ATTACK

BAÚ DO TESOURO

...



2

LISTA DE FIGURAS

Figura 1 – Contínua evolução das tecnologias de armazenamento de dados	10
Figura 2 – Interação e aplicação – SGBD – BD	12
Figura 3 – Exemplos de SGBDs mais conhecidos e utilizados	14
Figura 4 – Exemplos de tipos de dados numéricos	15
Figura 5 – Exemplos de tipos de dados em Strings	16
Figura 6 – Exemplos de tipos de dados de Data Hora	16
Figura 7 – Exemplos de tipos de dados Espaciais	16
Figura 8 – Acessando e saindo do MariaDB	18

LISTA DE CÓDIGOS-FONTE

Código-fonte 1 – Instalando o banco de dados MariaDB	18
Código-fonte 2 – Abrindo o MariaDB recém-instalado	18
Código-fonte 3 – Sintaxe de um CREATE DATABASE	19
Código-fonte 4 – Sintaxe de um CREATE TABLE	20
Código-fonte 5 – Sintaxe de um ALTER TABLE – uso do DROP ou do ADD.....	20
Código-fonte 6 – Sintaxe de um DROP TABLE	21

EMPRE

SUMÁRIO

INTRODUÇÃO A CODE REVIEW E BANCO DE DADOS.....	5
1 CONTEXTO	5
1.1 A eficiência do Code Review.....	5
2 INTRODUÇÃO AO CODE REVIEW.....	6
2.1 Instrumentalizando o Code Review.....	7
3 INTRODUÇÃO AO BANCO DE DADOS.....	9
3.1 Um breve histórico	9
3.2 Conceitos básicos	11
3.3 Sistemas de Banco de Dados	12
3.3.1 Vantagens do uso de SGBD	12
3.4 Modelagem de dados.....	13
3.4.1 Bancos de dados relacionais.....	14
3.5 Tipos de dados.....	15
4 INSTALAÇÃO E CONFIGURAÇÃO DO MARIADB	17
5 DDL – DATA DEFINITION LANGUAGE	19
5.1 Create Database	19
5.2 Create Table.....	19
5.3 Alter Table	20
5.4 Drop Table.....	21
REFERÊNCIAS.....	22

INTRODUÇÃO A CODE REVIEW E BANCO DE DADOS

1 CONTEXTO

A prática diária da programação e a familiaridade do programador com o código não o isentam de cometer erros. Muitas vezes, esses erros decorrem da modularidade do próprio projeto, tamanho ou até mesmo quantidade de programadores envolvidos, e não propriamente do desenvolvedor. São vários os motivos que podem acarretar defeitos de software, incluindo as várias fases do processo de desenvolvimento.

O aumento na demanda por projetos e a necessidade crescente de otimizar tempo e investimento fizeram crescer de importância o chamado CODE REVIEW, uma prática de verificação de código que visa reduzir defeitos de software, melhorando sua qualidade, efetividade e, conseqüentemente, sua manutenção.

1.1 A eficiência do Code Review

O Code Review se mostrou eficiente na mitigação de erros e tem sido aprimorado. Mas como podemos instrumentalizar essa prática e torná-la parte do projeto? É o que veremos neste e nos próximos capítulos desta disciplina.

Para tanto, é necessário conhecer as diversas interações de um sistema, seu banco de dados e funcionamento interno e ter arraigada uma forte noção de boas práticas de programação. Por fim, e não menos importante, conheceremos também os diversos tipos de falhas de segurança e vulnerabilidades que podem ser exploradas.

Esses incidentes de segurança geram prejuízos milionários às empresas e aos órgãos governamentais ao redor do mundo todo.

É impressionante a velocidade com que crescem os ataques e invasões a sistemas, explorando vulnerabilidades novas ou já conhecidas. Esses incidentes de segurança geram prejuízos milionários às empresas e aos órgãos governamentais ao redor do mundo todo. Um dos métodos atualmente mais utilizados se caracteriza

pela invasão e sequestro de dados, com posterior criptografia e pedidos de resgate vultosos. Iniciaremos nosso estudo por aí, não por acaso.

Vamos explorar algumas dessas vulnerabilidades, buscando, antes de tudo, entender a essência da programação: bancos de dados e as linguagens de programação propriamente ditas. A partir daí, poderemos melhorar nossa percepção de erros e suas correções. Vamos, primeiramente, entender melhor o surgimento e a operacionalização do Code Review.

2 INTRODUÇÃO AO CODE REVIEW

Mensurar a qualidade de um código não é tarefa das mais fáceis. Mas é fácil identificar um código de má qualidade: basta acompanhar os comentários dos usuários. Mas aí pode ser tarde demais: muito mais que a reputação pode ser perdida: pode envolver grandes somas financeiras, e ninguém quer isso para sua empresa ou cliente. É nesse contexto que o Code Review se insere, diminuindo drasticamente as chances de erros no código.

O Code Review normalmente é utilizado em projetos Open Source e também muito difundido por empresas de desenvolvimento de software. Nesta prática, alguns membros da equipe de desenvolvimento fazem a revisão do código do colega, antes que esse código seja incorporado ao projeto. O nível ou profundidade da verificação depende de vários fatores, como experiência individual e coletiva da equipe, especialidade de cada membro e nível de integração no projeto.

Mas, para além disso, o Code Review se mostra eficiente na detecção de erros e desvios das boas práticas, melhorando não apenas o resultado do projeto, mas também traz consigo uma série de benefícios. Entre eles, podemos destacar alguns:

- **Integração da equipe e compartilhamento de conhecimento:** a equipe passa a perceber o projeto como um todo, e como cada um é importante no processo. Além disso, o conhecimento é difundido entre os membros da equipe, o que é extremamente benéfico para todos.
- **Nivelamento do conhecimento:** à medida que o conhecimento é difundido, e que cada um tem a chance de explicar para os outros

membros os seus conhecimentos e experiências, a equipe se nivela, e todos ganham nesse processo.

- **Diminuição do retrabalho:** as chances de erros e bugs aparecerem no resultado final diminuem bastante, reduzindo na mesma medida a necessidade do chamado "retrabalho", economizando tempo e dinheiro.
- **Desenvolvimento de soluções alternativas:** conforme as revisões vão sendo feitas, pontos de vista e experiências diferentes têm a sua vez no projeto. Dá-se oportunidade a soluções alternativas que podem ter efeito potencializador no resultado final.
- **Aumento da atenção e senso de equipe:** como resultado de todos esses pontos positivos, a equipe amadurece e desenvolve um senso de equipe. Quando todos percebem a sua importância no projeto, ocorre até mesmo o aumento na atenção na hora do desenvolvimento.

Esse ciclo virtuoso resultante desses pontos positivos é extremamente benéfico e necessário. Mas como operacionalizar o Code Review e quais instrumentos podem ser utilizados?

2.1 Instrumentalizando o Code Review

Um dos maiores problemas das empresas desenvolvedoras ainda é o chamado "pai da criança". Esse termo designa o desenvolvedor que centralizou o projeto, ou um módulo desse projeto. A revisão de código pela equipe induz o colega à colaboração, evitando essa centralização danosa, que pode deixar códigos "órfãos" ou criar as chamadas "filas individuais", reduzindo a produtividade e criando gargalos desnecessários.

A revisão de código é um processo que evolui à medida que a organização envelhece e internaliza erros e acertos. E assim sendo, o processo de revisão vai se instrumentalizando de acordo com as peculiaridades de cada equipe e o seu nível de maturidade. Os métodos utilizados variam muito, mas seguem alguns preceitos comuns, conforme veremos abaixo. Importante perceber, apenas, que o processo de revisão de código é algo dinâmico e flexível, conforme o projeto demandar.

Existem, tradicionalmente, quatro maneiras de realizar revisões de código:

Revisões de código sobre o ombro: É a forma mais comum e fácil de revisar e não requer muito planejamento. Um membro mais experiente da equipe percorre o código do colega, na própria estação de trabalho do desenvolvedor, propondo melhorias e dando sugestões, tudo de maneira informal. Esse tipo de revisão pode ser feito juntamente com outros tipos e ferramentas colaborativas, o que atualmente é muito recomendado e utilizado.

E-mail Pass-Around: Esse método é amplamente difundido em distribuição geográfica mais ampla dos colaboradores. Algo muito comum nos dias de hoje devido às novas relações de trabalho a distância, por exemplo.

Nesse caso, um desenvolvedor dispara um e-mail para toda a equipe, contendo as modificações e comentários, normalmente por meio de sistemas de controle de versão (automatizando as notificações). Esse e-mail dá início à interação entre os membros, que apontam erros, solicitam alterações, tiram suas dúvidas etc. A similaridade com fóruns de discussão de código aberto não é mera coincidência, e a ideia de comunidade influenciou esse método de revisão.

Programação de pares: Nesse caso, a revisão de código é um processo contínuo, cujo feedback acontece em tempo real. Dois programadores utilizam a mesma estação de trabalho, porém um deles codifica, enquanto o outro dá o feedback em tempo real. Apesar de funcionar, esse método não é o mais eficiente, pois diminui a produtividade do revisor em outros processos.

Ferramentas assistidas: Consiste na utilização de ferramentas específicas que facilitam o processo de revisão, a conversa entre revisores, o histórico de modificações, entre outras funções igualmente eficientes. Esse tipo de ferramenta automatiza a revisão de código, deixando o desenvolvedor livre para se dedicar ao código.

Importante!

No desenvolvimento de softwares, há dois tipos de testes de código: dinâmico e estático. Enquanto o primeiro analisa o código levando em conta o conjunto e suas interações, o segundo é um teste feito pelo desenvolvedor após criar um novo código, antes de integrá-lo ao código completo.

Alguns exemplos de ferramentas de revisão de código: GitHub, Collaborator, Veracode, Reviewable, Review Board e Phabricator.

3 INTRODUÇÃO AO BANCO DE DADOS

Podemos não perceber, mas eles estão por todos os lados. Os Bancos de Dados são mais importantes nas nossas vidas do que a gente pode imaginar. Nossos dados estão praticamente todos lá: nosso nome, endereço, números de identidade e CPF, placa do carro e fotos. Até mesmo nossas preferências estão lá: histórico de pesquisas e de compras. Coisas bem importantes e sigilosas também estão lá: informações bancárias, dados de cartões de crédito etc.

Os Bancos de Dados estão nos bastidores, mas são a espinha dorsal dos sistemas. Tudo neles é importante: da forma com que os dados são armazenados à forma com que são acessados, do tipo de relacionamento ao tipo de uso que se fará deles. É como no teatro (para usar uma analogia bastante simples): toda a mecânica do espetáculo é suportada nos bastidores, e o que assistimos é resultado dessa interação harmônica.

3.1 Um breve histórico

O surgimento dos Bancos de Dados ocorre com a informatização das empresas, órgãos públicos e entidades, que viram na transição dos métodos tradicionais para informatizados a necessidade de agilidade no acesso aos dados, de segurança e de espaço para armazenamento destes dados.

Os armários de arquivos em papel, distribuídos em pastas e gavetas, demandavam grande espaço e tempo de organização, manutenção constante, sem falar na preocupação com a guarda física deles (incêndios, inundações etc.). Gradativamente, foram dando espaço aos meios de armazenamento informatizados, conforme as tecnologias foram sendo lançadas no mercado.

Com o rápido aumento na demanda, viu-se a necessidade de desenvolvimento de sistemas mais ágeis e disponíveis, que conversassem com o

banco de dados e garantissem a integridade deles. Tinham também que ser escaláveis na mesma medida com que estavam sendo demandados.

Surgiram, então, os **Sistemas de Gerenciamento de Banco de Dados (SGBDs)**, no início da década de 70, facilitando a programação de aplicações de bancos de dados. No começo, esses sistemas eram muito caros e difíceis de usar, necessitando pessoal especializado e bem treinado na utilização de um SGBD específico.



Figura 1 – Contínua evolução das tecnologias de armazenamento de dados
Fonte: Rezende (2006)

Como é natural em tecnologia, o aumento nos investimentos em pesquisa e desenvolvimento resultou no desenvolvimento de um tipo de SGBD que passou a dominar o mercado a partir da década de 80: o Relacional. Este tipo de SGBD virou, inclusive, padrão internacional, resultando num aumento gradativo nas técnicas e processos empregados em um projeto de banco de dados.

Com o advento da Internet na década de 90 e do modelo "cliente-servidor" já amplamente difundido, o banco de dados das organizações passou a estar conectado e disponível na rede a uma quantidade bem maior de clientes e usuários.

A virada do milênio veio, por sua vez, acompanhada do aumento no volume e quantidade dos dados armazenados nesses bancos de dados organizacionais. Demandou escalabilidade, dando espaço a uma nova forma de tratar esses dados e utilizando técnicas específicas de processamento: o chamado BIG DATA, conceito que descreve o grande volume de dados estruturados e não estruturados gerados a cada segundo.

Esse conceito cresce de importância e passa a marcar a disrupção que a década de 2010 causou em diversos mercados e a forma com que o cliente passou a interagir com os produtos e serviços que consome. A criação de novos produtos e soluções *On Demand* para clientes cada vez mais exigentes tem transformado a forma com que os dados são cruzados entre as diversas fontes de dados.

3.2 Conceitos básicos

Um Banco de Dados é um conjunto de dados armazenados e organizados de forma acessível, tempestiva, consistente e protegida. Esses dados armazenados agrupam informações com uma estrutura regular, inter-relacionada, e utilizada para uma mesma finalidade.

Esses dados são armazenados de forma organizada, ajudando a garantir a integridade, atomicidade, segurança, abstração dos dados, controle de inconsistências e redundância. Convém, ainda, fazermos a distinção entre três conceitos importantes para o nosso estudo: **dado**, **informação** e **conhecimento**.

- **Dado** é a matéria-prima da informação. São as informações em estado bruto: isoladamente, são apenas palavras. Exemplo: João Carlos.
- **Informação** são dados tratados e reunidos que passam a ter um significado mais concreto. A informação surge com o processamento dos dados. Exemplo: João Carlos é atleta de futebol.
- **Conhecimento** é o resultado do uso das informações e, como tal, pode ser utilizado, por exemplo, para tomada de decisões. Exemplo: João Carlos é atleta de futebol. Escolheremos ele para jogar no time principal da empresa este ano.

3.3 Sistemas de Banco de Dados

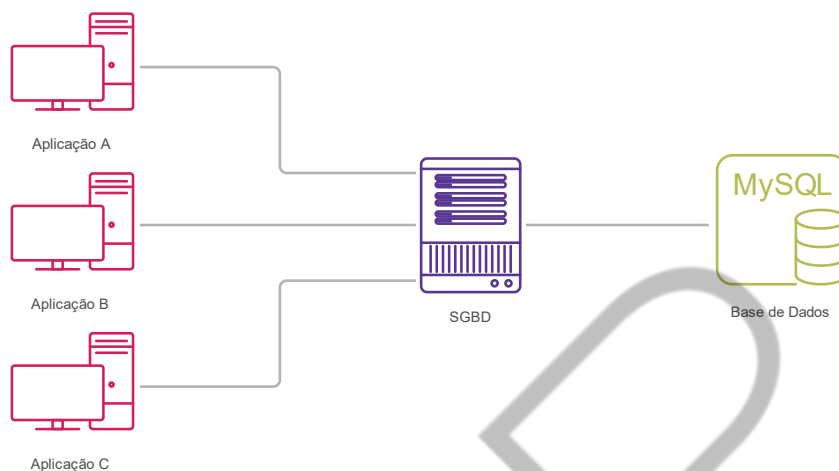


Figura 2 – Interação e aplicação – SGBD – BD
Fonte: Elaborado pelo autor (2020)

Normalmente, o termo Banco de Dados é usado como sinônimo de **SGBD**, Sistema Gerenciador de Banco de Dados; porém, o SGBD é um software que faz o gerenciamento do banco de dados, incorporando funções para a sua manutenção (automação de suas atividades de manutenção). O SGBD ajuda a garantir a independência dos dados em relação às aplicações, tornando-os independentes, podendo ser utilizados por várias aplicações ao mesmo tempo.

Ao isolamento do usuário dos detalhes internos do banco de dados, dá-se o nome de **abstração de dados**, que visa garantir apenas o necessário a ele: que os dados estejam disponíveis no momento em que são solicitados.

3.3.1 Vantagens do uso de SGBD

Podemos citar algumas vantagens dos SGBDs:

- **Escalabilidade:** Devido ao número elevado de transações e operações simultâneas, é uma das vantagens mais importantes, pois garante longevidade e efetividade à aplicação e ao banco de dados.
- **Padronização:** Aumenta a eficiência das aplicações e garante seu bom funcionamento. Estruturas e formatos de dados diferentes tornam a consulta mais demorada e dispendiosa.

- **Compartilhamento:** Simultaneidade de acesso a várias aplicações ao mesmo banco de dados.
- **Integridade dos Dados:** Validação dos tipos de dados que o SGBD faz antes de armazená-los. Isso garante que dados em formatos indevidos não sejam armazenados.
- **Segurança:** De grande importância atualmente, um SGBD possui entre outras, funcionalidades de criptografia e autenticação que visam evitar ataques de intrusão e de injeção (SQL Injection), por exemplo.

3.4 Modelagem de dados

Com o passar do tempo, os SGBDs passaram a utilizar diferentes formas de representação: os **modelos de dados**. Eles servem para descrever a estrutura das informações contidas em seus bancos de dados.

Os modelos de dados utilizados atualmente pelos SGBDs são:

- **Modelo Hierárquico:** Os dados são estruturados em hierarquias ou árvores. São estruturados em registro-pai e registros-filhos. Esse tipo de modelo só foi possível após a consolidação dos discos de armazenamento endereçáveis, que tornaram possível explorar essa estrutura de endereçamento físico dos discos para representar a estrutura hierárquica das informações.
- **Modelo em Redes:** Surgiu como uma extensão do modelo hierárquico, com o intuito de permitir envolver um mesmo registro em várias associações, organizadas em grafos.
- **Modelo Relacional** (mais utilizado atualmente): Sua estrutura fundamental é a relação (tabela). Mostrou-se mais flexível na recuperação dos dados, aumentando a independência dos dados nos SGBDs.
- **Modelo Orientado a Objetos:** Surge da necessidade de guardar estruturas de dados complexas (sistemas de informações geográficas - SIG e CAD, por exemplo) que demandam maiores limites de

armazenamento e representação semântica, e que passaram a ser um limitante no Modelo Relacional.



Figura 3 – Exemplos de SGBDs mais conhecidos e utilizados
Fonte: Elaborado pelo autor, adaptado por Fiap (2020)

3.4.1 Bancos de dados relacionais

Aprofundaremos nosso estudo no Modelo Relacional por ser mais utilizado atualmente. Esse modelo representa os dados contidos em um determinado banco de dados através de relações, e cada relação é uma tabela. Os nomes dos campos e das tabelas são importantes, porque indicam o que está sendo armazenado, a relação existente entre os dados e onde serão armazenados.

Chamaremos de **LINHA** cada registro dessa relação, e **ATRIBUTO** cada uma de suas colunas. **ESQUEMA** da relação, por sua vez, os campos existentes em determinada tabela, e **INSTÂNCIA** o conjunto de valores que cada atributo assume em determinado instante. A fim de evitar a duplicação das relações, é definido um atributo chamado **CHAVE PRIMÁRIA**, que definirá esse registro dentre uma coleção de registros.

Os comandos para manipulação e modelagem dos SGBDs terão sempre uma ou mais tabelas como base, por ser essa modelagem estruturada em tabelas. Entre as linguagens mais utilizadas para manipulação de um banco de dados relacional, a mais utilizada é a **SQL (Structured Query Language)**.

SQL é um conjunto de declarações utilizadas para acessar os dados utilizando Gerenciadores de Bancos de Dados. A maior parte dos SGBDs aceita a

SQL como linguagem-padrão para utilizar, administrar e manipular um banco de dados. Deste modo, utilizaremos essa linguagem no decorrer do nosso estudo.

3.5 Tipos de dados

Antes de iniciar a modelagem das tabelas de um banco de dados, é importante definir os tipos de dados que cada coluna das tabelas receberá. Essa escolha deve levar em consideração, principalmente: os tipos de valores que podem ser utilizados, o tamanho que podem ter, as operações que podem ser realizadas, os seus significados, se podem ser ou serão indexados, como deverão ser armazenados etc. Definir tudo isso é importante para que se consiga projetar um banco de dados o mais eficiente possível.

Alguns tipos de dados importantes:

Dados **NUMÉRICOS**: inclui todos os tipos numéricos possíveis – exatos, inteiros, aproximados, de ponto flutuante ou fixo etc. Alguns desses tipos de dados possíveis estão listados na tabela abaixo:

- TINYINT — número inteiro muito pequeno (*tiny*);
- SMALLINT — número inteiro pequeno;
- MEDIUMINT — número inteiro de tamanho médio;
- INT — número inteiro de tamanho comum;
- BIGINT — número inteiro de tamanho grande;
- DECIMAL — número decimal, de ponto fixo;
- FLOAT — número de ponto flutuante de precisão simples (32 bits);
- DOUBLE — número de ponto flutuante de precisão dupla (64 bits);
- BIT — um campo de um bit.

Figura 4 – Exemplos de tipos de dados numéricos
Fonte: Adaptado de Praciano (2014)

Dados em **STRINGS**: são cadeias de caracteres, podem ter qualquer conteúdo (de simples a binários), como arquivos e imagens, ser objeto de buscas e, ainda, ser comparadas. Alguns desses tipos de dados:

- CHAR — uma cadeia de caracteres (*string*), de tamanho fixo e não binária;
- VARCHAR — uma string de tamanho variável e não binária;
- BINARY — uma string binária de tamanho fixo;
- VARBINARY — uma string binária de tamanho variável;
- BLOB — um BLOB (*Binary Large Object* – Objeto Grande Binário) pequeno;
- TINYBLOB — um BLOB muito pequeno;
- MEDIUMBLOB — um BLOB de tamanho médio;
- LONGBLOB — um BLOB grande;
- TINYTEXT — uma string não binária e de tamanho bem reduzido;
- TEXT — uma string não binária e pequena;
- MEDIUMTEXT — uma string de tamanho comum e não binária;
- LONGTEXT — uma string não binária de tamanho grande;
- ENUM — é uma string, com um valor que precisa ser selecionado de uma lista predefinida na criação da tabela;
- SET — é um objeto que pode ter zero ou mais valores – cada um dos quais precisa ser escolhido de uma lista de valores predeterminados quando da criação da tabela.

Figura 5 – Exemplos de tipos de dados em Strings
Fonte: Adaptado de Praciano (2014)

Dados de **DATA HORA**: são muito importantes, inclusive, para acompanhar as mudanças ocorridas em um campo de uma tabela, por exemplo.

- DATE — o valor referente a uma data no formato 'CCYY-MM-DD'. Por exemplo, 1985-11-25 (ano-mês-dia). O 'CC' se refere aos dois dígitos do século (*Century*, em inglês);
- TIME — um valor horário no formato 'hh:mm:ss' (hora:minutos:segundos);
- TIMESTAMP — sequência de caracteres ou informação codificada que identifica uma marca temporal ou um dado momento em que um evento ocorreu. No MySQL, ele tem o formato 'CCYY-MM-DD hh:mm:ss' - neste caso, seguem a padronização ISO 8601
- YEAR — armazena um ano no formato 'CCYY' ou 'YY';

Figura 6 – Exemplos de tipos de dados de Data Hora
Fonte: Adaptado de Praciano (2014)

Dados **ESPACIAIS**: suporte a classes OpenGIS, que carregam valores geométricos simples.

- GEOMETRY
- POINT
- LINESTRING
- POLYGON

O GEOMETRY pode armazenar qualquer tipo de valor geométrico. Os outros valores simples (POINT, LINESTRING e POLYGON) têm seus valores restritos aos tipos geométricos a que se referem.

- GEOMETRYCOLLECTION
- MULTILINESTRING
- MULTIPOINT
- MULTIPOLYGON

Assim, GEOMETRYCOLLECTION pode armazenar coletâneas de objetos de qualquer tipo. Os outros tipos coletivos (MULTILINESTRING, MULTIPOLYGON e GEOMETRYCOLLECTION) restringem-se a cada forma geométrica particular.

Figura 7 – Exemplos de tipos de dados Espaciais
Fonte: Adaptado de Praciano (2014)

4 INSTALAÇÃO E CONFIGURAÇÃO DO MARIADB

Utilizaremos o Banco de Dados MariaDB em nosso estudo, servidor de banco de dados SQL robusto, rápido, multitarefa e multiusuário que pode ser usado em sistemas de produção de alta carga e missão crítica. É uma evolução *Open Source* do já consagrado MySQL, tendo sido criado pelos mesmos desenvolvedores do MySQL, que mantiveram a estrutura de código aberto, e é marca registrada da MariaDB Foundation.

Esse processo de cisão se deu após a compra do MySQL pela Oracle, o que causou preocupação de uma parte dos desenvolvedores e da própria comunidade software livre de que isso viesse a ocasionar dificuldades futuras. Iniciou-se, então, o desenvolvimento do MariaDB, cópia "quase-fiel" do MySQL.

O MySQL, na verdade, é hoje um programa de Licença Dupla, pois o usuário pode escolher entre utilizá-lo como Open Source ou comprar uma licença comercial padrão que dá direito a um suporte personalizado. Mais informações podem ser encontradas no site oficial do MariaDB (<https://mariadb.com/>) e do MySQL (<https://www.mysql.com/>).

Elaboramos um tutorial para a instalação e a configuração da conexão ao banco de dados MariaDB, que poderá ser utilizado para testes e execução de comandos SQL auxiliando no entendimento da sua dinâmica de funcionamento.

Iniciaremos a instalação do banco de dados MariaDB partindo do pressuposto de que já se possui familiaridade com Sistemas Operacionais Linux (utilizaremos a versão mais recente do Debian – 10.6.0 'Buster') e Softwares de Virtualização (no nosso caso, VirtualBox, na sua versão mais recente – 6.1.16). Seguem abaixo os links correspondentes às versões utilizadas neste tutorial, para efeito de instalação do ambiente de testes, caso ainda não as tenha.

<https://www.debian.org/distrib/>

<https://www.virtualbox.org/wiki/Downloads>

Para utilização do SU no Debian 10 'Buster', utilizaremos o comando < su - >.

Iniciaremos nosso tutorial pela instalação do banco de dados MariaDB no ambiente Debian criado no VirtualBox (versões utilizadas neste tutorial conforme mencionado acima) utilizando o seguinte comando:

```
$apt install mariadb
```

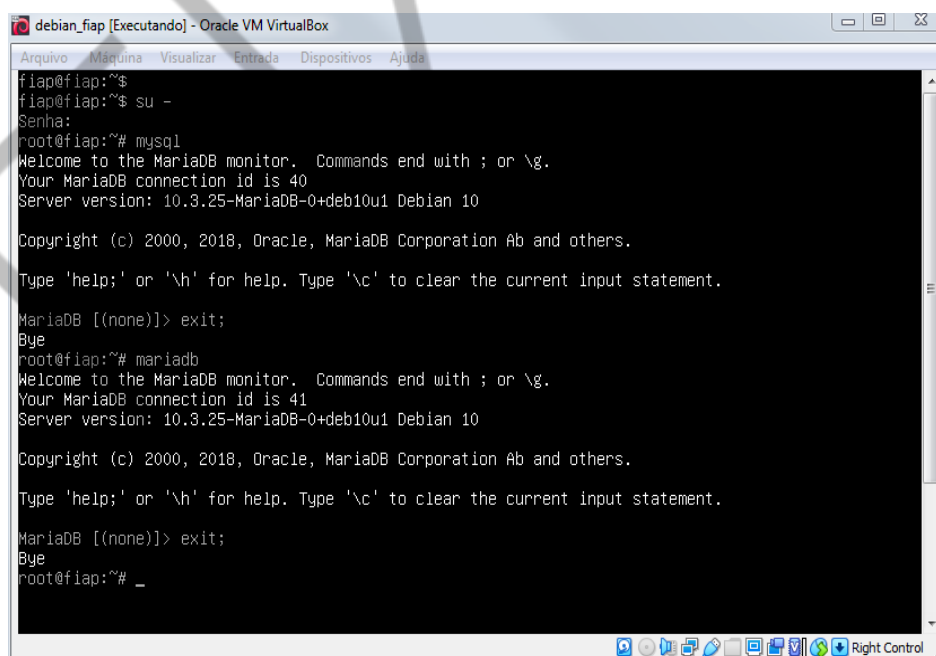
Código-fonte 1 – Instalando o banco de dados MariaDB
Fonte: Elaborado pelo autor (2020)

Após instalado, o comando abaixo abrirá o MariaDB para que possamos iniciar a modelagem e a manipulação dos dados:

```
$mysql;  
Ou:  
$mariadb;  
MariaDB [(none)]> exit;
```

Código-fonte 2 – Abrindo o MariaDB recém-instalado
Fonte: Elaborado pelo autor (2020)

Para sair da linha de comando do MariaDB, utilizaremos o comando **EXIT**, como mostrado acima. Os comandos no MariaDB (assim como no MySQL) são terminados com ";" (ponto e vírgula) ou "\g" (barra invertida + g).



```
debian_fiap [Executando] - Oracle VM VirtualBox  
Arquivo  Máquina  Visualizar  Entrada  Dispositivos  Ajuda  
fiap@fiap:~$  
fiap@fiap:~$ su -  
Senha:  
root@fiap:~# mysql  
Welcome to the MariaDB monitor.  Commands end with ; or \g.  
Your MariaDB connection id is 40  
Server version: 10.3.25-MariaDB-0+deb10u1 Debian 10  
  
Copyright (c) 2000, 2018, Oracle, MariaDB Corporation Ab and others.  
  
Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.  
  
MariaDB [(none)]> exit;  
Bye  
root@fiap:~# mariadb  
Welcome to the MariaDB monitor.  Commands end with ; or \g.  
Your MariaDB connection id is 41  
Server version: 10.3.25-MariaDB-0+deb10u1 Debian 10  
  
Copyright (c) 2000, 2018, Oracle, MariaDB Corporation Ab and others.  
  
Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.  
  
MariaDB [(none)]> exit;  
Bye  
root@fiap:~# _
```

Figura 8 – Acessando e saindo do MariaDB
Fonte: Elaborado pelo autor (2020)

Agora que nosso ambiente MariaDB está instalado e funcionando, precisamos conhecer mais alguns comandos e conceitos importantes antes de seguir com nosso *hands on*.

5 DDL – DATA DEFINITION LANGUAGE

A DDL (*Data Definition Language*) é a sintaxe de criação de estruturas de dados no SQL. Alguns comandos são essenciais na DDL, e devemos conhecê-los antes de iniciar a modelagem e a construção de um banco de dados relacional. São eles: **CREATE**, **DROP**, **ALTER**, **RENAME** e **TRUNCATE**.

5.1 Create Database

Para que seja possível adicionar e manipular dados, primeiro é necessário criar uma estrutura de banco de dados, utilizando, para isso, o comando **CREATE DATABASE**.

```
CREATE DATABASE <nome_do_banco>;  
USE <nome_do_banco>;
```

Código-fonte 3 – Sintaxe de um CREATE DATABASE
Fonte: Elaborado pelo autor (2020)

Esse comando apenas configura um local para adicionar as tabelas, ele não cria nenhum dado. A instrução **USE**, por sua vez, define o novo banco de dados criado como sendo o banco de dados padrão.

Veja mais sobre o comando Create Database na documentação do MariaDB:

<<https://mariadb.com/kb/en/create-database/>>

5.2 Create Table

A nossa próxima etapa é começar a criação de tabelas. O comando que realiza a construção de uma nova tabela, especificando as colunas e chaves (primárias e estrangeiras), é o seguinte:

```
CREATE TABLE <nome_da_tabela> (  
  <nome_da_coluna> <tipo-do-dado>,  
  <nome_da_coluna> <tipo-do-dado>,  
  <nome_da_coluna> <tipo-do-dado>  
) ;
```

Código-fonte 4 – Sintaxe de um CREATE TABLE
Fonte: Elaborado pelo autor (2020)

Importante!

É muito importante que se entenda o conceito de Chave Primária e Chave Estrangeira. Uma Chave Primária é como se fosse a coluna que cuida do endereçamento único de um registro nessa tabela. Enquanto a Chave Estrangeira, como o próprio nome diz, utiliza a chave de outra tabela como base.

Veja mais sobre o comando Create Table na documentação do MariaDB:

<><https://mariadb.com/kb/en/create-table/>

5.3 Alter Table

O comando **ALTER TABLE** modifica uma tabela por meio do acréscimo, da remoção ou da alteração de nome de coluna ou tipo de dado. Esse comando é muito completo e possibilita que sejam feitas todas as mudanças na tabela com apenas um comando. A desvantagem é o fato de que esse comando faz uma cópia completa da tabela. Se a tabela for muito grande, isso pode demorar.

```
ALTER TABLE <nome_da_tabela>  
  DROP COLUMN <nome_da_coluna>,  
  ADD <nome_da_coluna> <tipo-do-dado>  
;
```

Código-fonte 5 – Sintaxe de um ALTER TABLE – uso do DROP ou do ADD
Fonte: Elaborado pelo autor (2020)

Veja mais sobre o comando Alter Table na documentação do MariaDB:

<<https://mariadb.com/kb/en/alter-table/>>

5.4 Drop Table

O comando **DROP TABLE** remove completamente uma tabela, incluindo os registros contidos nela. Tenha muita cautela ao utilizar esse comando, pois ele não é reversível.

```
DROP TABLE <nome-tabela>;
```

Código-fonte 6 – Sintaxe de um DROP TABLE

Fonte: Elaborado pelo autor (2020)

Veja mais sobre o comando Drop Table na documentação do MariaDB:

<<https://mariadb.com/kb/en/drop-table/>>

Agora que vimos, neste capítulo, alguns comandos básicos e conceitos importantes sobre Code Review e Bancos de Dados, podemos nos aprofundar neste tema, explorando um pouco mais os recursos do Banco de Dados MariaDB.

REFERÊNCIAS

DAITYARI, S. Melhores ferramentas de Revisão de Código para Desenvolvedores. **Kinsta Blog**, 2020. Disponível em: <<https://kinsta.com/pt/blog/ferramentas-de-revisao-de-codigo/>>. Acesso em: 22 nov. 2020.

ELMASRI, R.; NAVATHE, S. B. **Sistemas de Banco de Dados: fundamentos e aplicações**. 6. ed. São Paulo: Pearson Addison-Wesley, 2011.

HEUSER, C. A. **Projeto de Banco de Dados**. 6. ed. Porto Alegre: Bookman, 2009. (Série Livros Didáticos, v. 4).

MACHADO, F. N. R. **Banco de Dados – Projeto e Implementação**. 2. ed. São Paulo: Érica, 2008.

MARIADB. **MariaDB Documentation: MariaDB Basics**. 2020. Disponível em: <<https://mariadb.com/kb/en/mariadb-basics/>>. Acesso em: 29 nov. 2020.

PAIVA, G. A importância do Code Review para a equipe de desenvolvimento de software. **Equals Lab**, 2018. Disponível em: <<https://medium.com/equals-lab/a-importancia-do-code-review-para-a-equipe-de-desenvolvimento-de-software-a47b70cb1560>>. Acesso em: 22 nov. 2020.

PRACIANO, E. **Tipos de dados do MySQL e MariaDB**. 2014. Disponível em: <<https://elias.praciano.com/2014/01/mysql-tipos-de-dados/>>. Acesso em: 29 nov. 2020.

QUESADA, A. *et al.* **Apostila de MySQL**. Coordenadoria do Governo Eletrônico (CGE) – Equipe de Treinamento Técnico, 2001. Disponível em: <http://www.telecentros.sp.gov.br/saber/apostilas/antigas/apostila_sql.pdf>. Acesso em: 29 nov. 2020.

REZENDE, R. **A história dos Bancos de Dados**. 2006. Disponível em: <<https://www.devmedia.com.br/a-historia-dos-banco-de-dados/1678>>. Acesso em: 25 nov. 2020.

SILBERSCHATZ, A.; KORTH, H. F.; SUDARSHAN, S. **Sistema de Banco de Dados**. 5. ed. Rio de Janeiro: Campus, 2006.